קטע קוד A

```
 1. class BaseExpression {
 2. public:
 3.   BaseExpression() {cout << '@';}
 4.   virtual ~BaseExpression() {cout << '#';}
 5.   virtual void print(ostream& out) const = 0;
 6.   friend ostream& operator<<
 7.   (ostream& out, const BaseExpression& e) {
 8.     e.print(out); return out;
 9.   }
10.   virtual double calc()const = 0;
11. };
12.
13. class Number: public BaseExpression {
14.   const double _d;
15. public:
16.   Number(double d) : _d(d) {cout << 'N';}
17.   virtual void print(ostream& out)const override { out << _d; }
18.   virtual double calc()const override {return _d;}
19. };
20.
21. class Expression {
22.   const BaseExpression* const _e; // const on content and pointer
23.   int* const _refCount;           // const only on pointer
24. public:
25.   Expression(BaseExpression* e)
26.     : _e(e), _refCount(new int(1)) {}
27.   Expression(double d)
28.     : _e(new Number(d)), _refCount(new int(1)) {cout << '!';}
29.   Expression(const Expression& e)
30.     : _e(e._e), _refCount(e._refCount) {++(*_refCount);}
31.   ~Expression() {
32.     if(--(*_refCount) == 0) {
33.       delete _e;
34.       delete _refCount;
35.     }
36.   }
37.   Expression& operator=(const Expression&) = delete;
38.   double calc() const { return _e->calc(); }
39.   friend ostream& operator<<(ostream& out, const Expression& e) {
40.     return out << *(e._e);
41.   }
42. };

    // code continues on next page
```

```
43. class BinaryExpression: public BaseExpression {
44.   const Expression _e1;
45.   const Expression _e2;
46.   const char _sign;
47. public:
48.   BinaryExpression(Expression e1, Expression e2, char sign)
49.     : _e1(e1), _e2(e2), _sign(sign) {cout << 'B';}
50.   virtual void print(ostream& out)const override {
51.     out << '(' << _e1 << ' ' << _sign << ' ' << _e2 << ')';
52.   }
53.   virtual double calc()const override {
54.     return calc(_e1.calc(), _e2.calc());
55.   }
56. protected:
57.   virtual double calc(double d1, double d2) const = 0;
58. };
59.
60. struct Sum: public BinaryExpression {
61.   Sum(Expression e1, Expression e2)
62.     : BinaryExpression(e1, e2, '+'){cout << 'S';}
63. protected:
64.   virtual double calc(double d1, double d2) const override {
65.     return d1 + d2;
66.   }
67. };
68.
69. int main() {
70.   Expression e1 = new Sum( 3.5, new Sum ( 2, 1 ) );
71.   cout << e1 << " = " << e1.calc() << endl;
72.   Expression e2 = new Sum( e1, 2 );
73.   cout << e2 << " = " << e2.calc() << endl;
74. }
```

נתון שהקוד עובר קומפילציה בהצלחה!

קטע קוד B

```
 1. class MyString {
 2.   int size;
 3.   int* refCount = nullptr;
 4.   char* str = nullptr;
 5.   void detach() {
 6.     if (--(*refCount) == 0) {
 7.       cout << "about to delete the string: " << *this << endl;
 8.       delete[]str;
 9.       delete refCount;
10.     }
11.   }
12.   void attach(const MyString& s) {
13.     size = s.size;
14.     refCount = s.refCount;
15.     str = s.str;
16.     ++(*refCount);
17.   }
18.   void copyOnWrite() {
19.     if (*refCount > 1) {
20.       cout << "about to do copy on write for: " << *this << endl;
21.       *this = str;
22.       cout << "after doing copy on write for: " << *this << endl;
23.     }
24.   }
25.   // inner private class
26.   class CharProxy {
27.     MyString& parent;
28.     const int index;
29.   public:
30.     CharProxy(MyString& p, int i) : parent(p), index(i) {}
31.     operator char() const { return parent.str[index]; }
32.     char operator=(char c) {
33.       parent.copyOnWrite();
34.       parent.str[index] = c;
35.       return c;
36.     }
37.   };
38.
39. public:
40.   MyString(const char* s)
41.   : size(strlen(s)), refCount(new int(1)), str(new char[size + 1])
42.   {
43.     strcpy(str, s);
44.   }
```

```
45.   ~MyString() { detach(); }
46.   MyString(const MyString& s) { attach(s); }
47.   MyString& operator=(const MyString& s) {
48.     if (this != &s) {
49.       detach();
50.       attach(s);
51.     }
52.     return *this;
53.   }
54.   CharProxy operator[](size_t i) {
55.     return MyString::CharProxy(*this, i);
56.   }
57.   char operator[](size_t i)const { return str[i]; }
58.   friend ostream& operator<<(ostream& out, const MyString& s) {
59.     return out << s.str
60.                << " (" << &s << " char* = " << (void*)s.str
61.                << ") -- refCount = " << *s.refCount;
62.   }
63. };
64.
65. int main() {
66.   MyString s1 = "Hello";
67.   MyString s2 = s1;
68.   cout << "1st char in s2: " << s2[0] << endl;
69.   cout << "s1: " << s1 << endl;
70.   cout << "s2: " << s2 << endl;
71.   s2[4] = '!';
72.   cout << "s1: " << s1 << endl;
73.   cout << "s2: " << s2 << endl;
74.   MyString s3 = "world?";
75.   cout << "s3: " << s3 << endl;
76.   s3[5] = '!';
77.   cout << "s3: " << s3 << endl;
78.   s3 = s1;
79.   s1[0] = 'h';
80.   cout << "s3: " << s3 << endl;
81. }
```

שימו לב:

נתון ששורה מספר 69 מדפיסה:

```
s1: Hello (0x7ffc7719c5a0 char* = 0x175cc40) -- refCount = 2
```

קטע קוד C – לשאלות הבונוס

```
class A {
    static A* first;
    static A* last;
    A *prev, *next;
public:
    A(): prev(last), next(nullptr) {
        if(last) last->next = this;
        last = this;
        if(!first) first = this;
    }
    A(const A&): A() {}
    A(A&& a): prev(a.prev), next(a.next) {
        if(&a == first) first = this;
        if(&a == last) last = this;
        if(a.prev) a.prev->next = this;
        if(a.next) a.next->prev = this;
    }
    A& operator=(const A&) {return *this;}
    ~A() {
       // print();
        if(prev) prev->next = next;
        if(next) next->prev = prev;
        if(first == this) first = next;
        if(last == this) last = prev;
    }
    static void printAll() {
        if(first) first->print();
        else cout << "list is empty";
        cout << endl;
    }
    void print()const {
        printSelf();
        if(next) next->print();
    }
    virtual void printSelf()const = 0;
};

class B: public A {
    int i;
public:
    B(int _i): i(_i) {}
    // ~B() { printAll(); }
    virtual void printSelf()const override {
        cout << i << ' ';
    }
};
```

```
class C: public B {
public:
    C(): B(7) {}
    ~C() { printAll(); }
    virtual void printSelf()const override {
        cout << "C: ";
        B::printSelf();
    }
};

A* A::first = nullptr;
A* A::last = nullptr;

int main() {
  B b1 = 5;
  B b2 = b1;
  b2 = 12;
  B b3 = std::move(b1);
  b3 = 99;
  C c1;
  cout << "end of main" << endl;
}
```

נתון שהקוד עובר קומפילציה בהצלחה!

בהצלחה !